MIDEL · NETWORK GUIDE

How the network works

Neuron, layer, block, head, teaching, and the other words on the screen, in plain language.

This guide is for the lab in this folder. It uses Midel's real sizes: a library of lessons, a small network of stored short lines, and a deep network saved as a file of weights. It also explains how lessons are added, how the deep network is kept up to date, and how an answer reaches the screen. The page does not train either network. No background in mathematics is required.

18,086
short lines in the small network

4,691
words the deep network knows

1.75 M
weights in the deep network

4,454
library lessons

Library	Short network	Deep network
A list of lessons	1 block, width 32	4 blocks, width 128
This usually writes the answer sentence.	Stored short lines sky, ice, grass.	Saved in a file. One lesson at a time.
They are not one set of weights.	*They are not one set of weights.*	*They are not one set of weights.*

Figure 1. Three pieces answer a question. Only two of them are neural networks.

CONTENTS

What is inside

01 The words that get mixed up
One table that sorts everyday words from lab words.

02 The path of one question
From the sentence you type to the answer on screen.

03 Tokens, vocabulary, embeddings
How text becomes numbers.

04 A neuron
Weight, bias, and ReLU with a worked sum.

05 A layer, and what width means
Width and depth are not the same.

06 A block
Residual, layer norm, and the mixer.

07 Heads and attention
Query, key, value, softmax, and the causal grid.

08 The deep network, end to end
One forward pass, step by step.

09 The short network and the deep network
Side by side, with live sizes.

10 Training, in one loop
Loss, gradient, learning rate, epoch.

11 Why a new lesson does not always enter the weights
Interference and the gates.

12 How an answer reaches the screen
Spelling help, matching, sources, Copy, Wikipedia.

13 Teaching: adding lessons
Short lines, full lessons, groups, and the writer.

14 Keeping the deep network up to date
The install queue, replay, skipped, relearn, and health checks.

15 Two builds: dev and deploy
What a hosted copy can and cannot do.

16 The side panel, card by card
Each drawing and what it shows.

17 Glossary
Every term used in this lab.

18 What to keep in mind
The short version.

4 October 2026. Open the page, ask something, and keep this beside it.

CHAPTER 01

The words that get mixed up

Most of the confusion is that everyday English and this lab reuses the same short words for different things. Read this page once. The rest of the guide is the longer version of the table.

Word on the screen	Plain meaning	In Midel
Neuron	One small decision: weigh inputs, add them, keep or silence the result.	You never see a single neuron. Width 128 means 128 of them side by side.
Weight	How much one input counts. Positive pulls toward, negative pushes away.	Learned numbers inside the file stage3_brain.js. Not the lesson text.
Layer	A row of neurons that read the same inputs together.	Inside a block there are layer-norm steps and the mixer. The buttons say Block, not Layer, for the stack.
Block	One full step: look back, mix, and add the result onto what came in.	Deep network: 4 blocks. Short network: 1 block.
Head	One way of looking back at earlier words.	Deep network: 4 heads. The panel shows which earlier word each head preferred.
Attention	How the 100% of 'looking back' is shared across earlier words.	The curves and the grid on the right. Darker or thicker means a larger share.
Token	A piece of text the network is allowed to hold.	A word, a number, or one of ? . ! , after lowercasing.
Embedding	The list of numbers that stands for one token before thinking starts.	128 numbers per token in the deep network.
Parameter	Any stored number the training is allowed to change.	Deep network: the embeddings, the block weights, and the final scores. About 1,745,408 numbers right now.
Context	How many recent tokens fit in one pass.	Deep network: 128. Short network: 64. Older tokens slide out of the window.

Figure 2. The same word, said carefully. Later sections draw each one.

> **A useful test.** If you can point to it in a lesson sentence, it is text. If it is a number stored in the brain file, it is a weight or another parameter. Sofia, as a fact, lives in the library. Sofia, as a token the deep network can write, lives only if that word is already in the vocabulary. This brain file has 4,691 entries.

CHAPTER 02

The path of one question

Before any neuron is involved, the page decides what kind of sentence you typed. The picture below is the whole product, not only the deep network.

1. **Read the sentence**
   A question asks. Typos are fixed when you press space.
2. **Split into tokens**
   Lowercase pieces: words, numbers, and the marks ? . ! ,
3. **Search the library**
   Lessons are scored by shared content words. Small typos still match.
4. **Strong match**
   At about 72% overlap, the stored lesson is the answer.
5. **Ask the deep network**
   It may continue only with words that already belong to that lesson.
6. **Hide a stray guess**
   If the guess leaves the lesson, the drawing is not shown beside the answer.

Figure 3. Six steps. The answer sentence usually comes from step 4, the library.

A question mark, or a sentence that starts with what, why, how, is, and similar words, is treated as a question. A short line, a question with an answer of one to three words or a statement of two to five words, is stored with the small network. A plain statement of about six or more words, with no question mark and not a request such as explain or list, is stored with the deep network's lessons. Neither write changes the deep weights by itself.

Yes or no is a special case. The page looks at the clause that actually mentions the subject. The deep network is allowed to say yes or no only when its vote matches that reading of the lesson. Otherwise you see the stored sentence.

CHAPTER 03

Tokens, vocabulary, embeddings

Token

A token is one piece the network can hold in its sequence. In this lab a token is a word, a number, or a single mark. The sentence is not sent in as a picture of letters. It is sent in as a row of tokens.

Tokenizer

The tokenizer is the cutting rule. Midel lowercases everything, removes commas that sit inside numbers, and then keeps numbers (including decimals and a trailing percent), words made of letters, and the marks ? . ! , A leading minus on a temperature is dropped. Letters outside plain English, such as n-tilde, are dropped, so a name can arrive shorter than you typed it.

Vocabulary

The vocabulary is the closed list of tokens the deep network has a row for. It began with the biology training and grows each time a library lesson is taught and the gates pass. This file has 4,691 entries. If a token is not on the list, it is dropped before the network runs. A name from a lesson that is still waiting can be missing. The network never receives that word, which is why some drawings used to wander.

Embedding

An embedding is the list of numbers that stands for one token at the start of the pass. In the deep network that list is 128 numbers long. The network does not store the spelling. It stores this list, and training moves the numbers. Two tokens can end up with similar lists if they were useful in similar places. That is a habit of the training, not a definition from a dictionary.

Positional encoding

Word order matters: capital of Bulgaria is not Bulgaria of capital. A second list of numbers, one per position, is added to the embedding so the first token is not interchangeable with the fourth. Those added numbers are the positional encoding. People also say position vector.

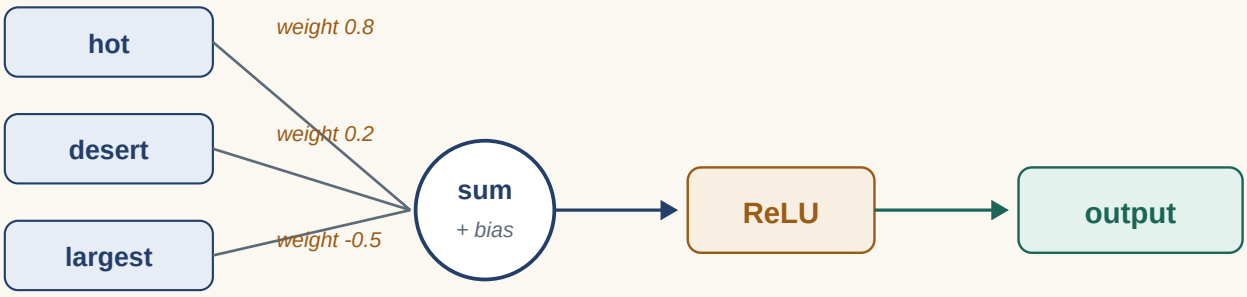
Context

Context, or the context window, is how many tokens fit in one pass. The deep network keeps the most recent 128. The short network keeps 64. Anything older falls out of the window and cannot be looked at on that pass.

CHAPTER 04

A neuron

A neuron is one small decision. It is also called a unit. It is not a copy of a biological cell. The picture uses three toy inputs so the arithmetic is visible. A real layer has many more inputs, and you do not edit them by hand.



Negative weight: this input counts against the sum. ReLU: a negative sum becomes silence (zero).

Figure 4. One neuron. Each input is scaled by a weight, the results are added, a bias is added, then ReLU keeps or silences the sum.

Weight

A weight is the number on one arrow in that picture. Multiply the input by the weight. A weight of 0.8 means this input counts a lot. A weight of -0.5 means this input counts against the total. A weight near zero means this input barely matters. Training's whole job is to change weights.

Bias

The bias is a number added after the weighted inputs, before ReLU. It shifts the decision. With a bias, the neuron can output a positive number even when every input is zero.

Activation and ReLU

The activation is the rule applied to that sum. ReLU, short for rectified linear unit, is the rule inside Midel's mixer. If the sum is negative, the output is 0. The neuron stays silent. If the sum is positive, the output is the sum itself. Silence is how a network ignores a combination that did not matter.

Worked sum. Suppose the inputs are 1, 1 and 1, with weights 0.8, 0.2 and -0.5, and the bias is 0.1. The sum is 0.8 + 0.2 - 0.5 + 0.1 = 0.6. ReLU leaves 0.6 as it is. If the sum had been -0.2, ReLU would have output 0.

CHAPTER 05

A layer, and what width means

A layer is a whole row of neurons that read the same inputs together and produce a new row of numbers. One neuron makes one number. A layer of 128 neurons makes 128 numbers. That count is the width. In papers it is often called d_model.

One layer = many neurons reading the same inputs at once



Width of the deep network is 128. People also call this d_model. The short network's width is 32.

Figure 5. Width. The deep network carries 128 numbers per token. The short network carries 32.

Depth is a different idea. Depth is how many steps sit on top of each other. The deep network is deep because it stacks 4 blocks, not because 128 is a large number. A wide network and a deep network are not the same phrase.

You will also meet the word layer inside a block: layer norm is a tidy-up step, and the MLP is made of layers. When the side panel says Block 1, Block 2, Block 3, Block 4, it means the four stacked steps, not four single neurons.

CHAPTER 06

A block

A block is one complete step of a transformer. It is much larger than a neuron. Each block does two jobs, and it keeps a copy of its input so it can add that copy back at the end of each job. The deep network runs four blocks in a row. The output row of numbers from block 1 is the input to block 2.

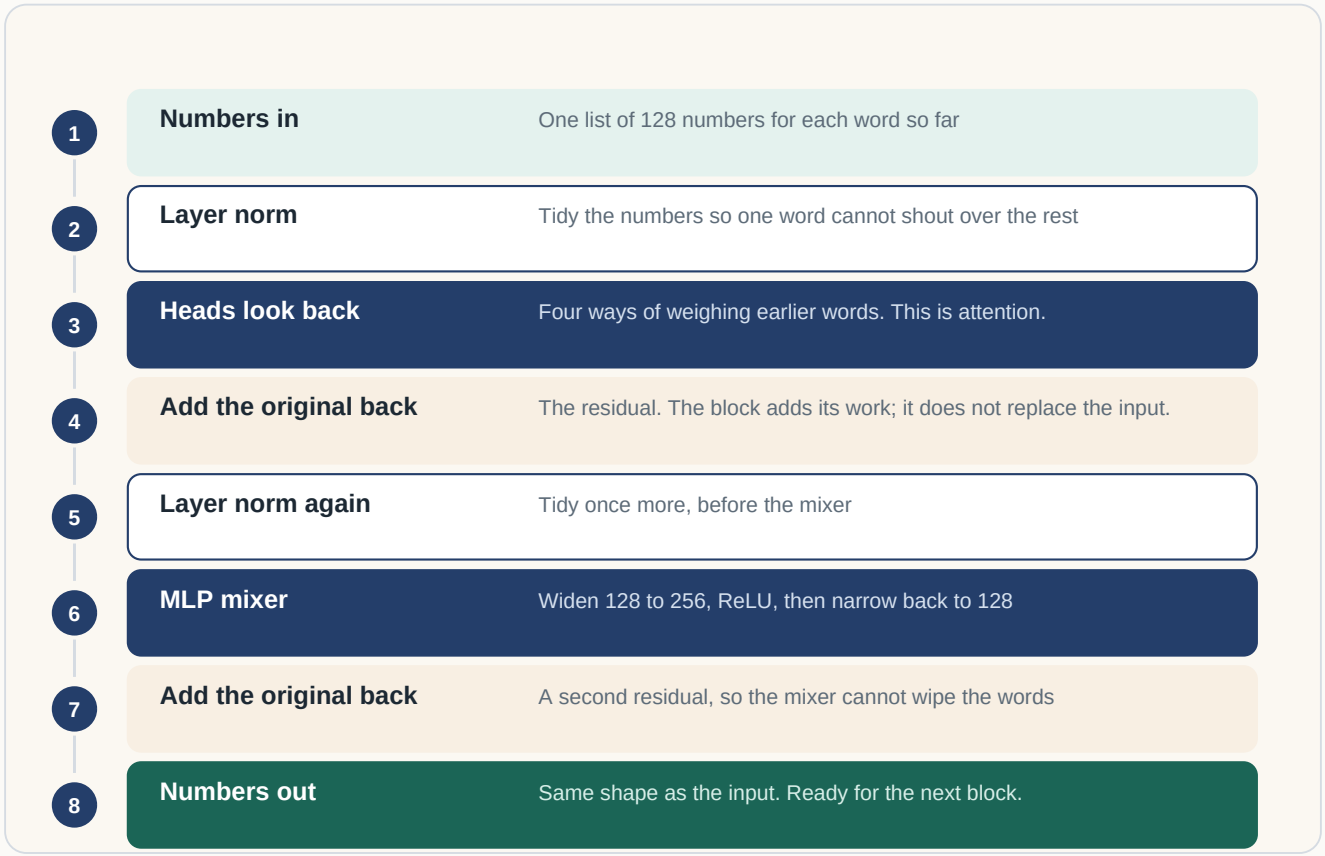


Figure 6. Inside one block. Steps 4 and 7 are residual connections: add the new work onto the numbers that entered, instead of replacing them.

Residual, or skip connection

The residual is that addition. If a head is unsure, its contribution can be small, and the numbers that entered the block still pass through. The network can stack several blocks without each one being forced to rebuild the sentence from nothing.

Layer norm, and pre-norm

Layer norm recenters and rescales one token's numbers so a few huge values do not dominate the next step. Pre-norm means the tidy-up happens first, then the work, then the residual add. That is the layout of the deep network. You do not set these numbers yourself.

MLP, or feed-forward mixer

After the heads look across words, the MLP looks at each word's own 128 numbers. It widens them to 256, applies ReLU, and narrows them back to 128. MLP means multi-layer perceptron, which is an older name for a small stack of neuron layers. Feed-forward means this part does not look at other positions. The looking-across is the attention's job, not the mixer's.

CHAPTER 07

Heads and attention

Attention is the looking-back. For the token the network is finishing, every earlier token gets a share of the attention, and those shares add up to 100 percent. A head is one such look. The deep network has four heads, so it can keep four opinions about what mattered. They are combined before the residual add.

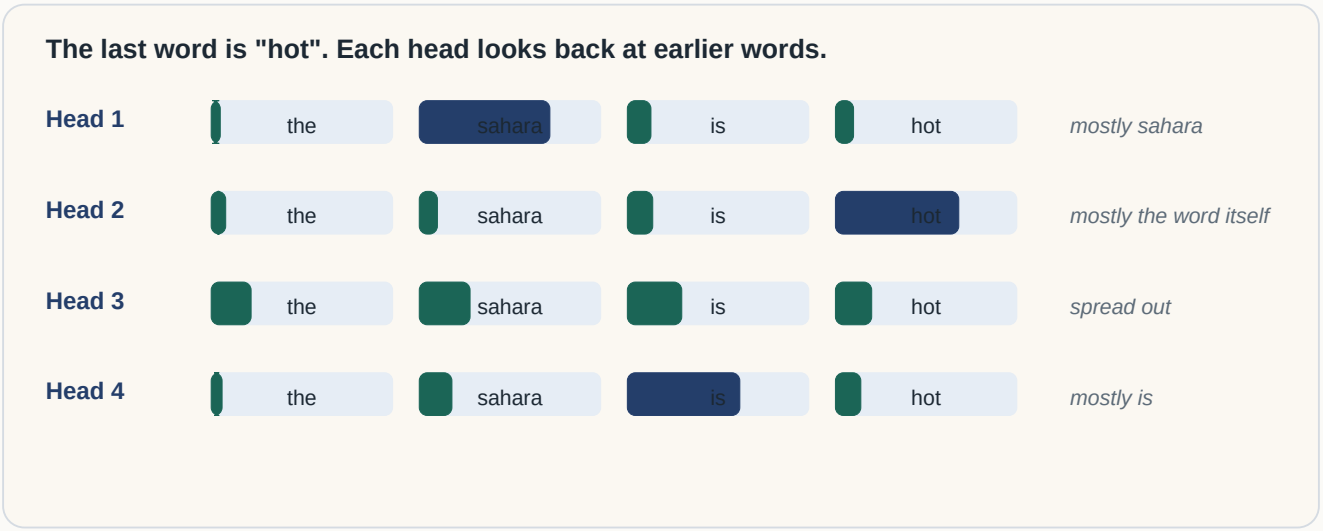


Figure 7. Four heads on the same four words. The bars are invented numbers to show the idea. A real pass uses the weights, and the shares change with the sentence.

Head size is the width of one head's private numbers. The deep network splits its 128 numbers into 4 heads, so each head is 32 wide. The short network has one head, and its whole width is 32, so that single head is also 32 wide.

Query, key, and value

Inside one head, each token is turned into three lists. The query is what this token is looking for. The key is what each earlier token offers as a label. The value is the information that gets mixed in if the query and the key match. You do not need to compute this to read the panel. The panel already shows the result: the shares.

Softmax

Softmax turns a list of raw match scores into shares that add up to 1. A score that is much larger than the others takes most of the share. The percentages on the heads and on the next-word bars are this kind of share.

Causal

Causal means the share on future tokens is forced to zero. While the network is at the word is, it may look at the and sahara and is. It may not look at hot, because hot has not been written yet. The grid shows that as empty cells.

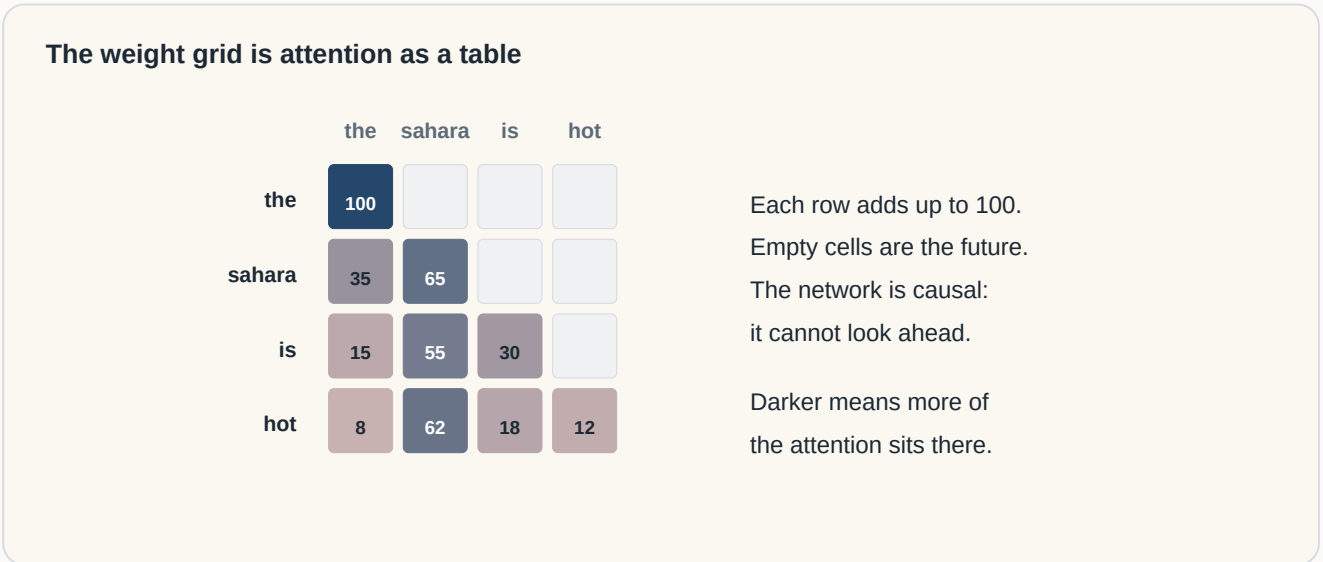


Figure 8. A causal grid for four words. Read across a row to see where that word's attention went. Empty cells are the future. The bottom row is what the panel means by the last word.

CHAPTER 08

The deep network, end to end

This is the forward pass of the deep network. Nothing in this picture changes a weight. It only reads the current weights and writes one next token. The example is a capital question. The chat sentence comes from the library. The network can continue it only with tokens it already has.

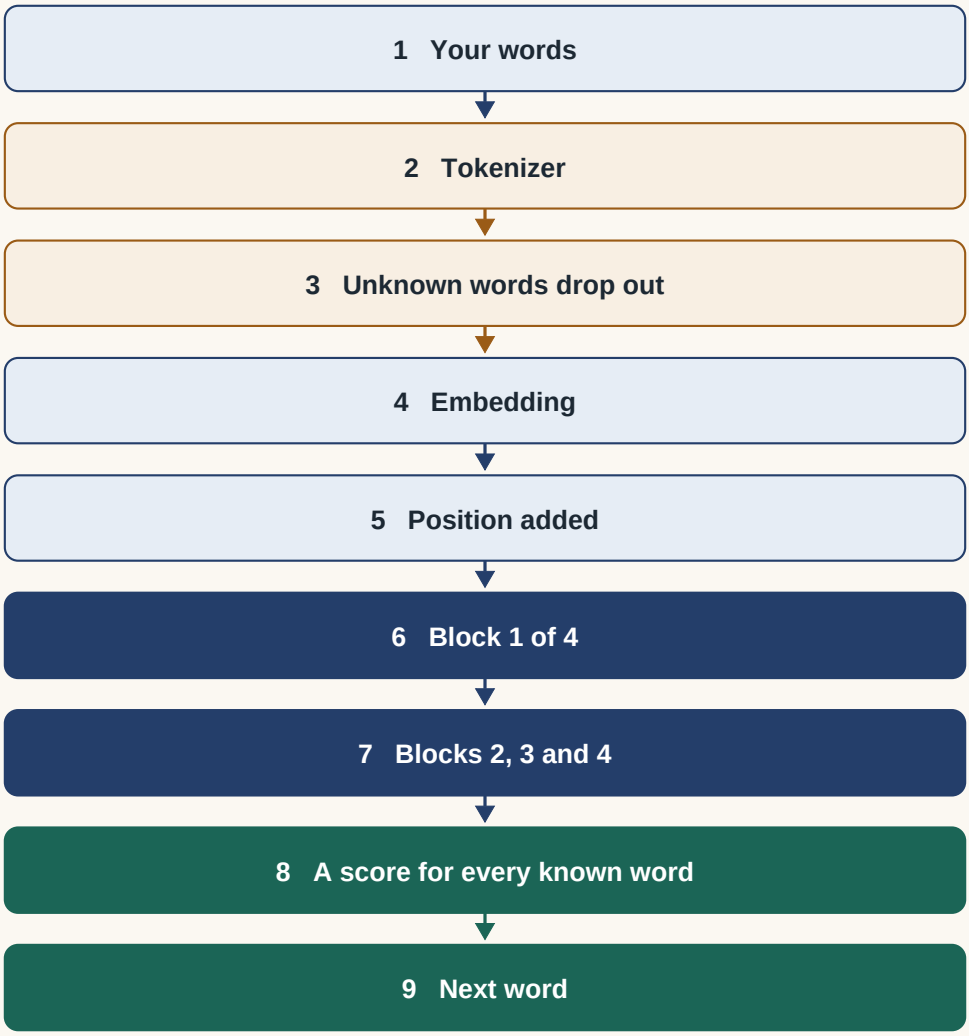


Figure 9. One forward pass. Steps 6 and 7 are the four blocks from Figure 6. Step 9 can repeat: the new word is appended, and the pass runs again.

Logits and probability

At step 8 the network produces one raw score per vocabulary token. Those raw scores are the logits. Softmax turns them into probabilities, which are the bars in Next word the network preferred. The tallest bar is the greedy choice.

Greedy decoding

Greedy means: take the single highest probability, write that token, and run the forward pass again with the new token on the end. The network does not draft several full sentences and then choose. If the first token is the wrong city, the rest of the sentence often follows that wrong city, because each new pass is continuing what was just written.

Hidden state

After each block, every token still has 128 numbers, but they are no longer the raw embedding. They have been mixed with other tokens through attention and with the MLP. That mixed list is the hidden state. It is the block's working memory for those tokens.

> **What happened with Bulgaria.** The library contains the lesson that Sofia is the capital, and that lesson has since been taught into the weights, so bulgaria is on the list. Before that lesson was taught, step 3 dropped the name and the forward pass continued from older habits. The page still shows the stored sentence, and it hides a free guess that has left the lesson.

CHAPTER 09

The short network and the deep network

Both are transformers. They are not two views of one brain. A lesson you add goes into one of them: a short line into the small network, a full lesson into the deep network's lesson file. The page does not train either one in the browser.

	Short network	Deep network
Where it lives	Lessons in short_added.js (loaded on the first question)	Weights in stage3_brain.js, lessons in the library
Blocks	1	4, pre-norm
Width	32	128
Heads	1	4, each of size 32
Context	64 tokens	128 tokens
Mixer	A small MLP	MLP, width 256, then ReLU
What it practices	111 built-in short lines plus 17,975 added lines in 26 groups	One waiting library lesson at a time, replayed with up to 64 recent ones. A save is kept only when the gates pass. 929 lessons are installed.
When weights change	Not in the browser. Lessons are stored as text.	One lesson at a time, only if the gates pass.
Rough size	About 753,056 weights now. The count grows with the words in the stored lines. Vocabulary 11,605	1,745,408 weights, 16.6 MB, vocabulary 4,691

Figure 10. Two networks. The library is a third piece and is not in this table, because it has no weights.

CHAPTER 10

Training, in one loop

Training is the only time weights change. The page does not run this loop. The deep-network trainer does, for one waiting library lesson at a time. The picture is that one step.

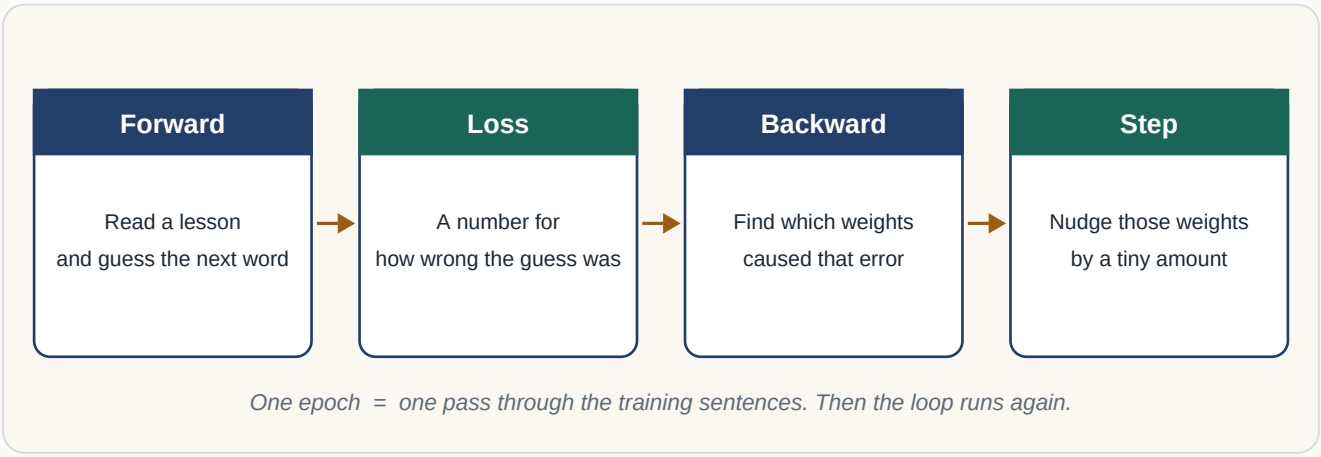


Figure 11. One training step. An epoch is this picture repeated until every practice sentence has been used once. Then a new epoch starts.

Loss

The loss is a single number for how surprised the network was by the correct next token. If the correct token already had almost all the probability, the loss is small. If the correct token had almost none, the loss is large. Training tries to push the loss down. It does not try to memorize a sentence as a paragraph you can scroll. It tries to make the next token less surprising, one position at a time.

Gradient and backpropagation

The gradient of one weight is 'if I increase this weight a tiny bit, does the loss go up or down, and by how much?' Backpropagation is the method that computes that for every weight, starting at the loss and walking backward through the blocks. The step then moves each weight a little in the direction that reduces the loss.

Learning rate

The learning rate is the size of that little move. A large rate learns fast and can wreck older answers. A small rate is gentler and may not pick up a new name before training ends. The from-scratch deep run starts at 0.003 and slowly shrinks. Teaching one lesson starts tiny (0.0001), warms up to 0.0006 over two passes, then eases down along a curve. The short network uses 0.005.

Epoch

An epoch is one pass through the whole set of practice sentences. A from-scratch deep run is built for 80 epochs. The page does not run those passes. More epochs are not automatically better. After some point the network is fitting the practice wording rather than the idea.

Optimizer, Adam, and SGD

The optimizer is the rule that turns a gradient into a step. SGD, stochastic gradient descent, takes a plain step. Stochastic means the sentences arrive in a shuffled order. Adam keeps a running memory of recent gradients and of how jumpy each weight has been, and it takes smaller steps on jumpy weights. The deep network uses Adam. Parts of the short network use a clipped form of SGD, which also limits how large any single step can be. That limit is gradient clipping.

Seed

The seed is the start of the random number generator. Embeddings begin as small random numbers. Seed 7 makes that start repeatable. It does not encode facts.

Parameter

A parameter is any number the optimizer is allowed to change: one entry of an embedding, one weight in a head, one bias in the mixer. The deep network's parameters are exactly the contents of the brain file. The lesson sentences are not parameters.

Inference

Inference is a forward pass used to answer, with the optimizer switched off. Every question you type is inference. The page does not train. The separate deep-network trainer is the training step, and only when you run it.

Checkpoint, fine-tune, and absorb

A checkpoint is a saved brain file. Fine-tuning means more training that starts from a checkpoint instead of from random numbers. Absorb is the fine-tune that reads every lesson file in this folder and tries to teach those lessons to the installed deep network. It writes a new brain file only if the old checks still pass.

CHAPTER 11

Why a new lesson does not always enter the weights

The library can hold a new fact immediately, because a library is a list. A network can hold a new fact only by moving weights until the forward pass prefers the right next token. Those are different kinds of memory.

Many geography lessons share a shape: a city name, then is the capital of, then a country. The shared words are easy. The one token that makes Bulgaria different from Afghanistan is easy to mix up, because the sentences are almost the same. During a fine-tune the network often learns the shape and then attaches the wrong name. That mix-up is called interference. A stronger learning rate, or more epochs, can also move the old biology answers until a yes becomes a no.

The checks that refuse a bad save are called gates. Angel Falls must still continue with the number 979. The Sahara must still be hot. Antarctica must still not be called the largest hot desert, and must still be the largest desert. Mercury must still be closer to the Sun. If a gate fails, the previous brain file stays. The new lessons remain in the library, which is why the chat can still say Sofia.

A new lesson is never practiced alone. Each run replays up to 64 of the most recently installed lessons, plus the yes and no checks, so the new name does not push them out. After a few passes the trainer checks **retention**: the new needle word, the new lesson, and every replayed lesson must still come out right. If one slipped, training goes on, and a short rescue rehearses the dropped lesson before the run gives up. A needle is a distinctive word taken from the opening of the answer, so a glue word such as the or of never counts.

> **What you should expect.** A short line is stored with the small network at once. A full lesson is stored with the deep network's lessons at once. The deep weights change only when that lesson's training run finishes and the gates pass. Until then, the chat still uses the stored sentence, and the side diagram will not pretend a free guess is the lesson.

CHAPTER 12

How an answer reaches the screen

Chapter 2 drew the path in six steps. This chapter is what you actually see in the thread: what helps you type, which blocks can appear, and what the small links under them do.

Spelling help in the prompt

The prompt checks spelling against the lessons, the notes you added, and the deep network's words. A finished word that is one or two letters off a known name is replaced when you type the space. After two characters, up to five lines appear: a corrected or completed word first, then matching questions. Click one, or highlight it and press Enter, to ask it. A word row replaces only the word you are typing. Escape closes the list.

Typo-tolerant matching

Matching is forgiving on purpose. Every word of the question is compared with the ask words (what, who, where) and with each lesson's own vocabulary. Missing letters, a dropped first letter, and small swaps are allowed. The allowance grows with word length: up to two edits on typical words, three on long ones. If two candidates tie, nothing is guessed. When typos helped but the overlap is under about 72%, the chat shows **Did you mean...?**, and a click fills the prompt.

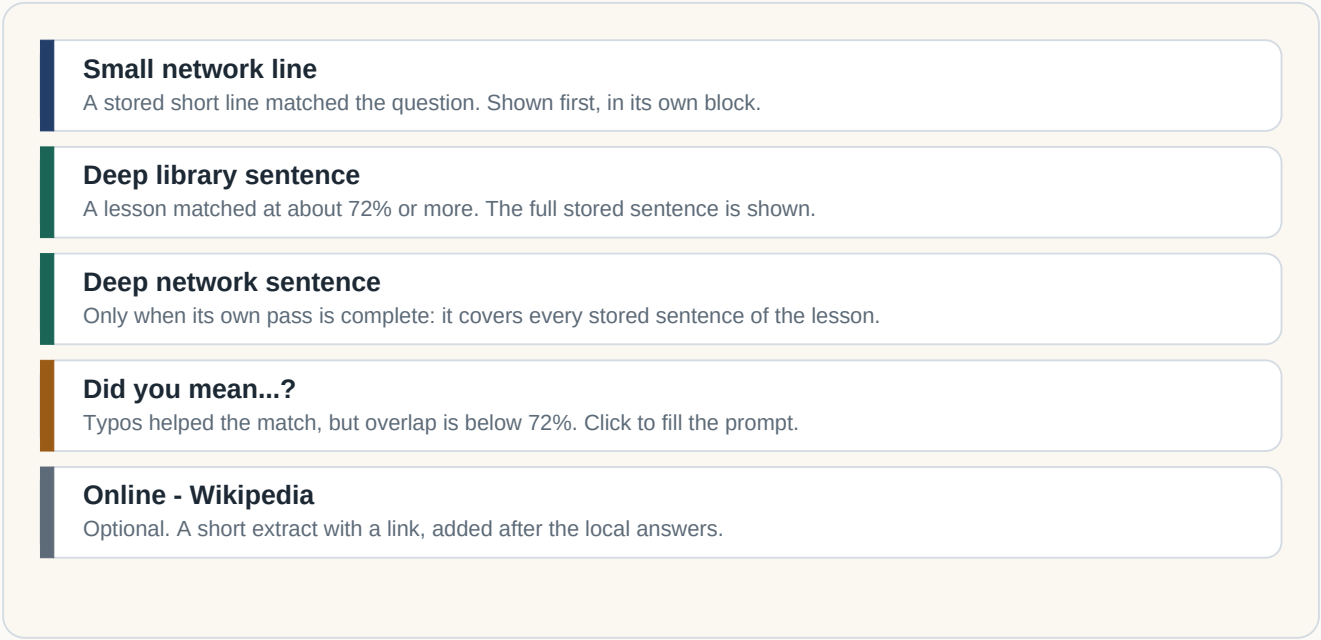


Figure 12. The blocks an answer can have, from top to bottom. Most questions show one or two of them.

Which sentence is shown

When one lesson matches well, the chat may show a sentence the deep network wrote itself, but only if that pass is **complete**: it used the same question forms as training, stayed on the lesson's words, contains the key answer word, and covers every stored sentence of the lesson. A partial paraphrase is never shown. The full stored text is shown instead. Yes or no answers, and questions where two lessons disagree, use the library sentence, or the network's yes or no when it agrees.

A shorter question that is an exact start of a lesson's question counts as the same lesson. If the small network also holds a short line for it, both answers appear: the short line and the full stored sentence.

Source lines, Copy, and Ask Gemini

Under an answer, a source line names where it came from, such as *from Science #3208*. A row saved from Wikipedia reads *Deep network learned #id*. A weak, shared-word-only hit is labelled *Loose library match* until a better row replaces it. Each answer block has its own **Copy** button that copies only that block. **Ask Gemini** opens Google AI Mode for the same question. While a reply is finishing, three breathing dots sit where it will start.

Online lookup and Remember

In the left panel, **Use Wikipedia summaries in chat** adds a short English extract, labelled *Online - Wikipedia*, with a link. On the Mac that owns the files it asks the writer program first. On a hosted copy the browser loads Wikipedia directly. The extract is licensed CC BY-SA. A second switch, **Remember**, exists only in the dev build. When the deep library did not really answer, it saves one trimmed lesson of one or two sentences, in the category Wikipedia, if the article passes the relevance gates. Weights are not trained by that save.

> **Remember the difference.** Showing a Wikipedia paragraph and teaching the network are two separate things. The paragraph is read from the web. A saved row becomes a library lesson at once, and reaches the weights only when the install loop in chapter 14 gets to it.

CHAPTER 13

Teaching: adding lessons

Teaching is the pencil icon on the left rail. It is how new knowledge is written down. What you type is sorted by its shape, and the Add box uses the same sorting as the prompt.

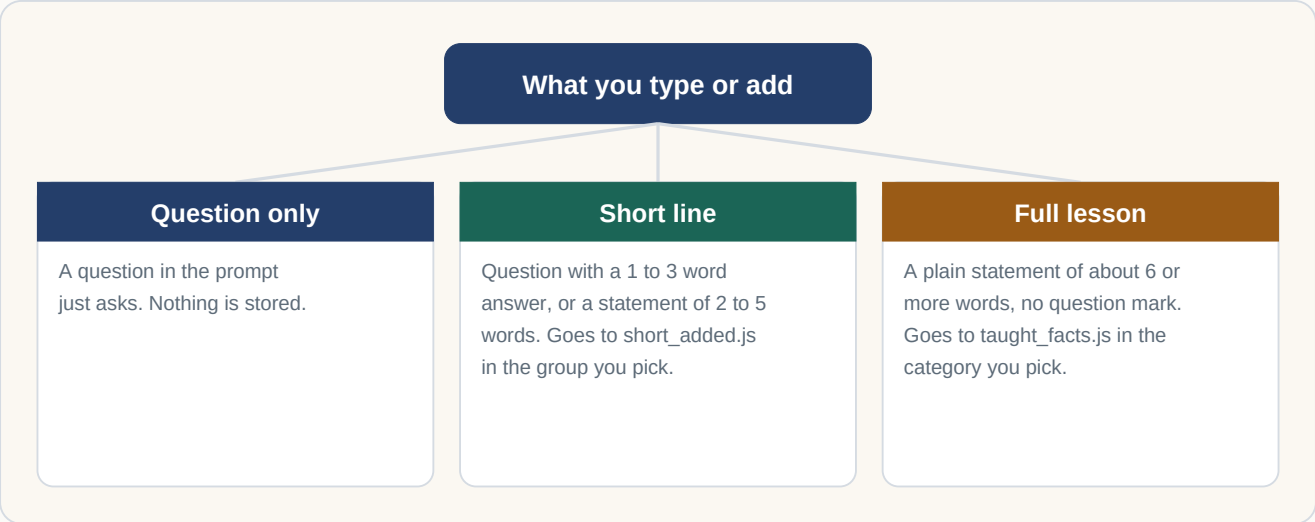


Figure 13. Where a line goes. Only the second and third branches write anything.

Groups

A short line goes into one of 26 groups in `short_added.js`, such as Animals, Sky and space, or Money and shops. A full lesson goes into a category section of `taught_facts.js`. Type a new category name and it is added to the list. Statement lines also answer as questions: a line such as *Honey never spoils.* is also matched by *what is honey*. Those question forms are built in memory and are never written to the file.

The writer

The page cannot write files. A small local program, the **writer** (`save_taught.py`, port 8787, this computer only), does it. Add and Remove are saved to disk first, and the page keeps the lesson only after that succeeds. On a Mac it is installed once and keeps running after login. On Windows it is started and left open. A static host cannot run it, so adding lessons works only on the computer where the writer runs.

Ids and corrections

Every full lesson has an id. Teaching rows count up from 3001 and skip the blocks owned by packs, for example 4001 to 4040 for gemstones. Saves merge by id, so a page with an old copy of the list cannot erase newer rows. **Correct answer** in the dev chat changes one row, and it can queue that lesson to be relearned by the deep network. An audit command checks for id collisions and stray gaps.

Packs

A **pack** is a file of lessons on one subject, named `something_facts.js`: biology, math, gemstones, chemistry, geography, skills, artificial intelligence, design, Bulgarian history, and scientists. With the science and geography lessons, that makes 962 built-in lessons. Teaching rows (3,492 now) are counted separately. A new pack file is picked up automatically, and it must use its own block of ids.

A large file, loaded late

The short-line file is big. The page does not read it at start-up. A tiny companion file, `short_added.meta.js`, carries the counts so the Network load card is right from the first second. On the first question the page fetches the full file, builds a word index, and then searches only about 160 candidate lines plus the 111 built-in ones.

CHAPTER 14

Keeping the deep network up to date

A lesson is usable in the chat the moment it is saved, because the library is just a list. Reaching the deep network's weights is slower and is done by a separate trainer, one lesson at a time.

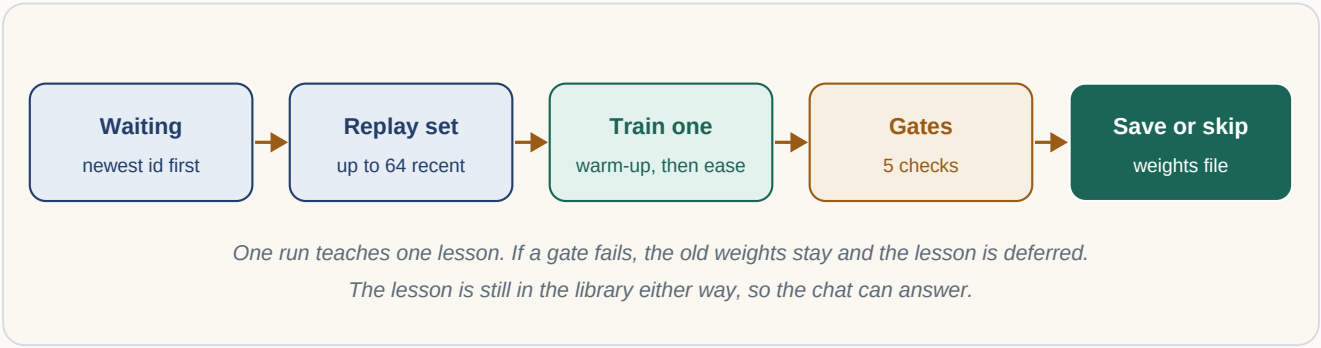


Figure 14. One install run. The same loop repeats until the queue is empty.

The queue

Only a lesson that has its own question and answer can be taught. A note with no question stays in the chat. Each run takes the highest id that is not yet installed or skipped, so a high pack id can go before an older Teaching row. The trainer prints a queue line such as *installed 920, skipped 7, pending, teachable*. A loop command repeats runs until the queue is empty.

Installed, skipped, deferred, relearn

Installed means the weights learned it and every check passed. **Deferred** means a run failed; it is retried after the other waiting lessons, and after four failures it becomes **skipped**. A skipped lesson stays in the library for chat. You can also skip one by hand. **Relearn** is a second queue for lessons whose text was corrected, or that drifted: the trainer teaches the new text and exports only if the checks still pass. Currently 929 lessons are installed.

Health check

A read-only **eval** command loads the weights and re-asks every installed lesson, plus the Sahara, Mercury, and Angel Falls gates. It writes nothing. A low score is common because its needle test is stricter than the chat, which shows the full library sentence. Lessons that drifted are candidates for relearn, not proof that the library is wrong.

Wikipedia batch

A batch command reads a long list of questions, fetches a summary for each, and saves rows that pass the same relevance gates as Remember. It only grows the lesson file. The install loop is what moves them into the weights, and the two can safely run side by side. Two writers on the same port, or editing the weights file by hand while a loop runs, is not safe.

Backup

One **backup** command refreshes the numbers in the model notes, rebuilds this guide, and packs the two zip files. Wait until the weights file ends with a semicolon before running it, so a half-written file is never packed.

CHAPTER 15

Two builds: dev and deploy

The same page ships in two ways. The **dev** zip is the full lab for another Mac. The **deploy** zip is a static folder for a web host. Chat, answers, and the side panel work the same in both.

	Dev build	Deploy build
Left panel	Full Teaching: Add form, lesson lists, Remember switch	About Midel: network load and the Wikipedia display switch
Add or remove lessons	Yes, with the writer running	No. The page is read-only
Remember Wikipedia	Yes, dev only	No
Wikipedia display	Through the writer, or straight from the browser	Straight from the browser
Short-line counts	short_added.meta.js	Counts are embedded in the page itself
Contains	Page, packs, trainer, writer, scripts, notes, this guide	Only what the page loads: page, packs, weights, logos

Figure 15. The two builds. After changing lookup or page code, run the backup again and upload the new page from the zip.

CHAPTER 16

The side panel, card by card

The panel on the right is a view of the latest question. It is not a second brain. The cards, from top to bottom:

- **Path.** The route the page took: the words, whether the library matched, whether the network was allowed to replace the sentence, and what was shown.
- **Lessons considered.** The closest lessons and the overlap score. The one marked used is the lesson behind the answer.
- **Attention.** The last step of a pass that stayed on the lesson. The stored sentence is the answer above this card. The drawing has just read one word of that sentence and is choosing the next. The curves are earlier words in the same sentence. The circle is the word just read. The footer names the next word. This is not a score of how well the question matched the lesson. If the free guess left the lesson, this drawing is hidden on purpose.
- **Signal.** The same deep pass, played back. Columns are the tokens. Rows are the 128 units. The frames run from the tokens entering, through the four blocks, then the preferred next words. Brighter means a larger activation.

- **Heads on the last word.** One row per head. The word and the percentage are that head's largest share. Four rows means the deep network. One head means you are looking at the short network.
- **Library-only notice.** When a lesson is in the library but its topic words are not in the brain file yet, the panel says so instead of drawing a guess. The drawings return after that lesson is taught.
- **Next word the network preferred.** The probability bars from step 8 of Figure 9. The top bar is the greedy choice. It is a guess about the next token, not a definition.

The Layer buttons switch which of the four blocks you are looking at. Layer 1 is the first look. Layer 4 is the last look before the next word is chosen. They do not retrain anything. The animation on the curves is only a way to see the same shares move. The numbers are the pass. The motion is not a separate computation. The sizes of both networks are on the Teaching panel, on the Network load card.

CHAPTER 17

Glossary

Every term used while building and running this lab, in one place. If a sentence in a later session uses one of these words, this is the meaning it has here.

Term	What it means here
Activation	The rule that turns a neuron's sum into its output. Midel's mixer uses ReLU: negative becomes zero, positive passes through.
Adam	The optimizer used for most of the deep network. It remembers which weights have been jumpy and takes smaller steps on those.
Attention	A share of 100% across earlier tokens. The network is deciding which previous words matter for the word it is about to write.
Backpropagation	The backward pass. After the loss is known, the trainer works out how each weight contributed, so it knows which way to nudge.
Bias	A number added inside a neuron or layer before the activation. It lets the unit fire even when the inputs are near zero.
Block	One full transformer step: tidy, look back with the heads, add that onto the input, tidy, mix with the MLP, add again. Deep network: 4. Short network: 1.
Causal	The network may look at tokens it has already read, and not at tokens that would come later. The empty corner of the grid is that rule.
Checkpoint	A saved copy of the weights. The installed copy is stage3_brain.js. A lesson that fails the gates does not replace it.
Context	The maximum number of tokens in one pass. Deep: 128. Short: 64. This is also called the context window or max sequence length.
Coverage	The library's match score: how many of the question's meaningful words appear in a lesson. A strong answer needs about 0.72 or more, and more than half the content words.
Curriculum	The list of short sentences stored with the small network. The page answers from those lines. It does not train them in the browser.
Deep network	The wider transformer whose weights live in stage3_brain.js. Four blocks, width 128, four heads. Its vocabulary grows as library lessons are taught.
Deferred	A lesson whose install failed. It is retried after the other waiting lessons and becomes skipped after four failures.
Embedding	A learned list of numbers for one token. Similar use in training can pull embeddings closer, but closeness is not a dictionary definition.
Epoch	One full pass over the training sentences. A from-scratch deep run uses 80. Teaching one library lesson uses a short run: 2 warm-up passes, then up to 16, plus a rescue phase of up to 12 if an older lesson slips. The page does not run these passes.

Term	What it means here
Feed-forward	Another name for the MLP mixer inside a block. It looks at each word's numbers on their own, without looking back at other words.
Fine-tune	More training that starts from weights you already have, instead of starting from random numbers. Teaching one waiting lesson is this kind of run.
Forward pass	Reading tokens from left to right and producing the next-word scores. Asking a question uses a forward pass. It does not by itself change any weights.
Gate	A check that must pass before new deep weights are saved. Examples: Angel Falls still produces 979, and the Sahara and Mercury yes/no answers stay correct.
Gradient	The direction and size of 'this weight made the loss worse.' The step moves the weight the opposite way.
Gradient clipping	A safety limit so one wild sentence cannot shove the weights by a huge amount in a single step.
Greedy decoding	Always write the single most likely next token, then repeat. The network does not plan a whole sentence and then pick the best one.
Head	One attention look-back. Four heads read the same tokens and are allowed to care about different earlier words. Head size here is 32, because 128 / 4 = 32.
Hidden state	The numbers that stand for the tokens after a block has mixed them. Still 128 per token in the deep network.
Inference	Using the network to answer. No weights change. The opposite of training.
Layer	A row of units that transform the numbers once. Layer norm is a layer. The MLP contains layers. A block is several layers wrapped together.
Layer norm	A tidy-up that recenters and rescales a word's numbers. Pre-norm means this tidy-up happens before attention and before the mixer.
Learning rate	How large each nudge is. The deep from-scratch run starts at 0.003. A fine-tune uses a smaller rate so old answers move less.
Library	The searchable lessons: science, biology, math, gemstones, chemistry, geography, skills, artificial intelligence, design, Bulgarian history, and scientists, plus the Teaching rows you add. This is not a neural network.
Library-only	A lesson that chat can show but whose topic words are not in the brain file yet. The side panel says so instead of drawing a guess.
Logits	The raw scores for the next word, before they are turned into percentages. Bigger means more preferred, but they are not yet probabilities.
Loss	One number that says how surprised the network was by the correct next word. Training tries to make this number smaller.
Meta file	short_added.meta.js. A tiny file with the short-line counts, so the Network load card is right before the big file is fetched.
MLP	Multi-layer perceptron. The small mixer inside each block: a wide layer (256), ReLU, then back to 128.
Needle	A distinctive word from the opening of a lesson's answer. A saved lesson must still produce it. Glue words such as the or of never count.
Neuron	One unit. It multiplies inputs by weights, adds a bias, and applies an activation. Also called a unit or a cell. The home screen draws decorative neurons. The side panel shows attention and activations, not one circle per weight.
Optimizer	The rule that turns gradients into a step. Adam for most deep-network weights. Clipped SGD for some short-network weights.
Overfitting	The weights get very good at the practice sentences and worse at a new wording of the same idea.
Pack	A file of lessons on one subject, named something_facts.js, with its own block of ids.
Parameter	A stored number training can change: a weight, a bias, or an embedding entry. More parameters can hold more patterns, until the patterns collide.
Positional encoding	Numbers added so position 1 is different from position 4 even when the word is the same. Without them the network would see a bag of words.

Term	What it means here
Pre-norm	Layer norm is applied before each sub-step inside the block, and a residual adds the result back. This is the deep network's layout.
Probability	A share between 0 and 1. The next-word bars are probabilities. They add up to 1 across the whole vocabulary.
Queue	The lessons waiting to be taught to the deep network. Highest id first.
Relearn	A second queue for lessons whose text was corrected or that drifted. The trainer teaches the new text and exports only if the checks pass.
ReLU	Rectified linear unit. If the sum is negative, output 0. If it is positive, output the sum. Used in the MLP.
Remember	A dev-only switch that saves a trimmed Wikipedia lesson when the library did not really answer.
Replay	Practicing up to 64 recently installed lessons together with a new one, so the new one does not push them out.
Residual	A skip connection. The block's output is 'what came in, plus what this step added.' Information can pass through even if a head is unsure.
Retention	After a few passes, the new lesson and every replayed lesson must still come out right. Otherwise training continues, then a rescue rehearses what slipped.
Seed	The starting number for random draws. Seed 7 makes a fresh training run begin the same way each time. It does not fix the meaning of a word.
SGD	Stochastic gradient descent. A plain step along the gradient. 'Stochastic' means the sentences are taken in a shuffled order, not as one giant sum.
Short network	The tiny transformer described in the page. One block, width 32, one head, context 64. Its lessons are stored with the page. A new short line is written into a chosen group in short_added.js. The page does not train it in the browser.
Skipped	A lesson the weights could not learn without breaking an older answer. It stays in the library.
Softmax	The function that turns logits into probabilities that add up to 100%. A large logit takes most of the share.
Teaching row	A full lesson saved from the Add box, from Remember, or from a Wikipedia batch. Ids start at 3001.
Token	One item in the sequence: a word, a number such as 3.14 or 979, or a mark such as ?
Tokenizer	The rule that cuts text into tokens. Midel lowercases, strips commas inside numbers, and keeps words, numbers, and ? . ! ,
Training	Changing weights so the next-word guesses get less wrong. Asking a question in the chat is not training.
Transformer	The kind of network Midel uses. Its main trick is attention over the tokens already read, stacked in blocks.
Typo match	Matching that tolerates missing letters, a dropped first letter, and small swaps, with an allowance that grows with word length.
Vocabulary	The list of tokens the network has a row for. The deep list grows when a library lesson is taught. A token outside that list is dropped.
Weight	A single learned number that scales one connection. The lesson text is not a weight. The file of weights is the brain file.
Width	How many numbers represent one token. Deep network: 128. Short network: 32. Also written d_model.
Writer	save_taught.py. The small local program that saves Teaching to disk. It listens on port 8787 on this computer only.

Figure 16. The working vocabulary of this lab. The same words in another product can be sized differently. The ideas match.

CHAPTER 18

What to keep in mind

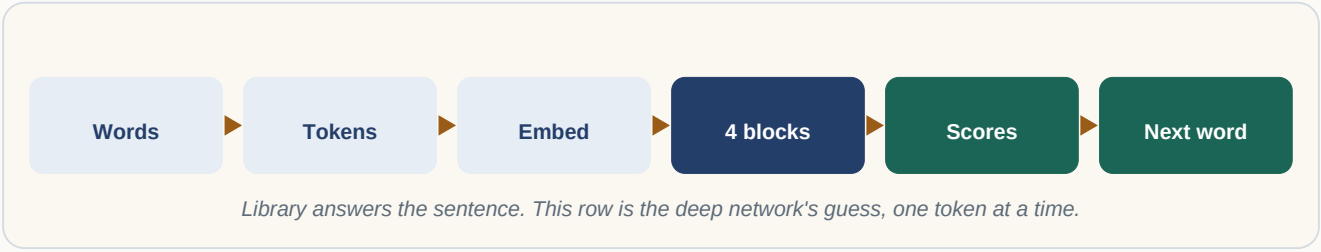


Figure 17. The whole deep pass in one row. Pair it with Figure 3 when you want the page's full path, including the library.

- The chat sentence comes from the library when the match is strong enough.
- The short network and the deep network are two transformers. Training one leaves the other as it was.
- A neuron is one decision. A layer is a row of them. A block is look-back plus mixer. A head is one way of looking back.
- Attention is a split of 100 percent over earlier tokens. The grid's empty cells are the future, which a causal network cannot see.
- A token missing from the vocabulary is dropped. The deep network cannot attend to a word it never received.
- Asking is a forward pass. Weights move only during training, and a deep-network save is kept only when the gates pass.
- A lesson is usable in chat the moment it is saved. It reaches the weights only when the install loop teaches it and the gates pass.
- Adding lessons needs the writer program, so it works only on the computer where it runs. A hosted copy is read-only.
- A Wikipedia paragraph on screen is read from the web. A saved Wikipedia row is a library lesson.

When a word on the screen is still unclear, look it up in the glossary, then find its picture: the neuron in Figure 4, the block in Figure 6, the heads in Figure 7, the full pass in Figure 9, the training loop in Figure 11.